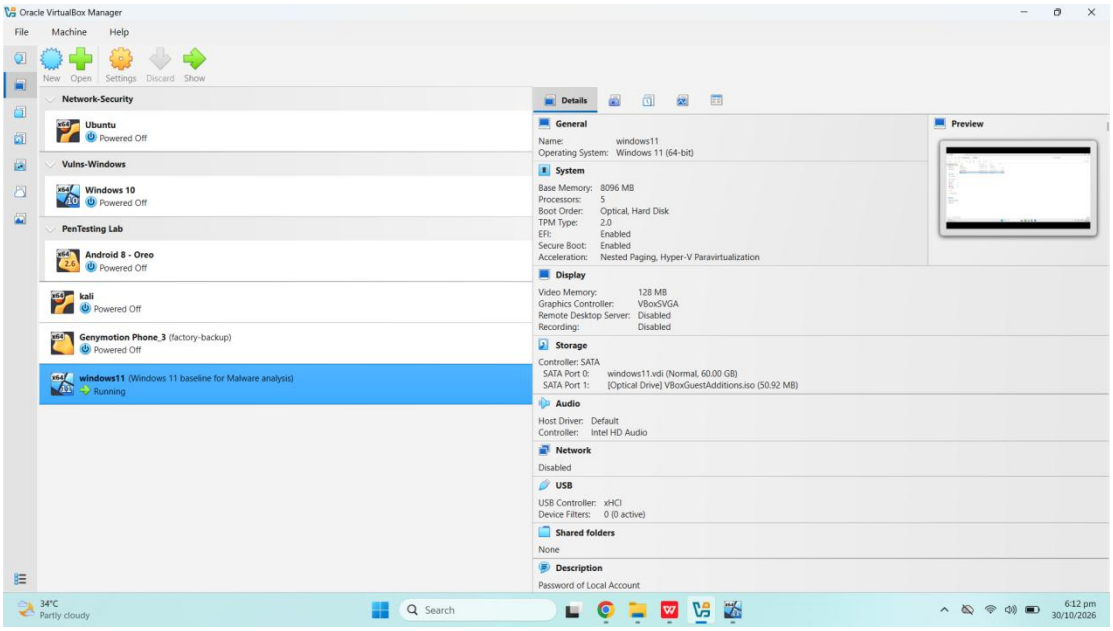
1) Executive Summary

This report presents a malware analysis and threat investigation of the selected SalatStealer sample using static and dynamic analysis techniques. Static analysis tools, including Detect It Easy (DIE), PEStudio, are used to examine the executable's structure, compiler information, imported functions, strings, and potentially suspicious indicators without executing the sample. The investigation aims to observe the sample's behavior in an isolated virtual environment, identify Indicators of Compromise (IOCs), assess potential security risks, and recommend appropriate detection and mitigation measures. Findings will be documented using screenshots and technical evidence to support the final threat assessment and improve understanding of malware detection and incident response.

2) Lab Environment and Tools Used

A controlled and isolated virtual environment was established to perform static and dynamic malware analysis. The laboratory environment was configured using Oracle **VM VirtualBox** with a **Windows 11** virtual machine as the analysis system. The purpose of this isolated environment was to minimize potential risks to the host operating system and prevent accidental exposure to external networks during malware execution.

2.1) Virtual Machine Configuration



In the above figure it is shown that The following configuration was used to establish the malware analysis environment.

Component	Configuration
Virtualization Software	Oracle Virtual box 7.20
Host Operating System	Windows 11
Guest Operating System	Windows 11
Processor Allocation	5 Processors
Memory Allocation	8GB RAM
Virtual Disk	60 GB
TPM Type	2.0
Video Memory	128 MB
Shared Folder	Disabled
Clipboard	Disbaled
Graphics Controller	VBoxSVGA
Internet Connectivity	Disblaed using static Analysis

2.2) Tools Used

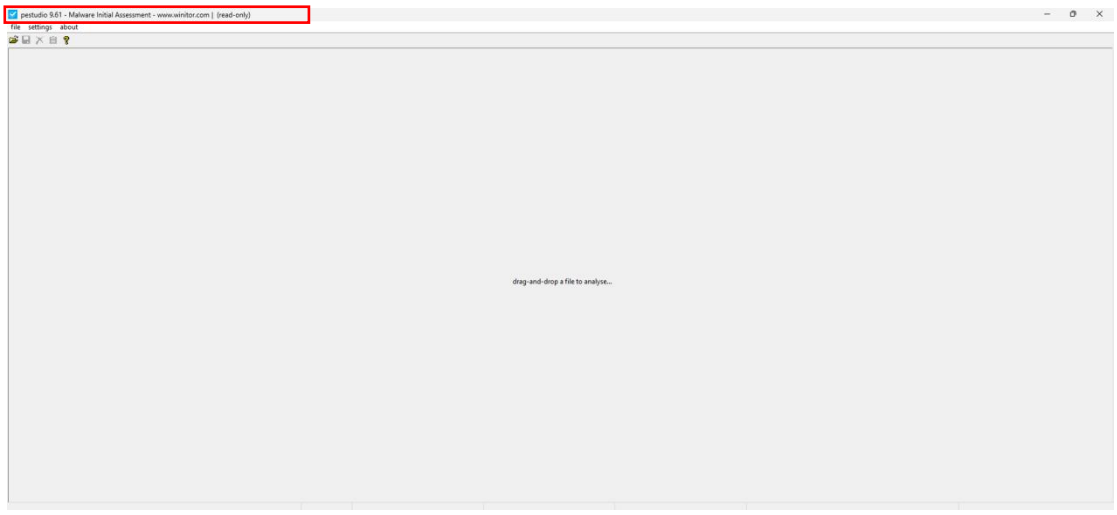
1) Static Analysis Tools

The tools that has been used for **static analysis** here are Detect it easy and Pestudio.

1) To download Pestudio visit the website **softpedia.**

Pestudio: Uses during static analysis to perform initial malware assessment by inspecting PE file headers, imported libraries, suspicious API calls, embedded resources, and hidden metadata without executing the binary.

Link: **https://www.softpedia.com/get/Programming/Other-Programming-Files/PeStudio.shtml**

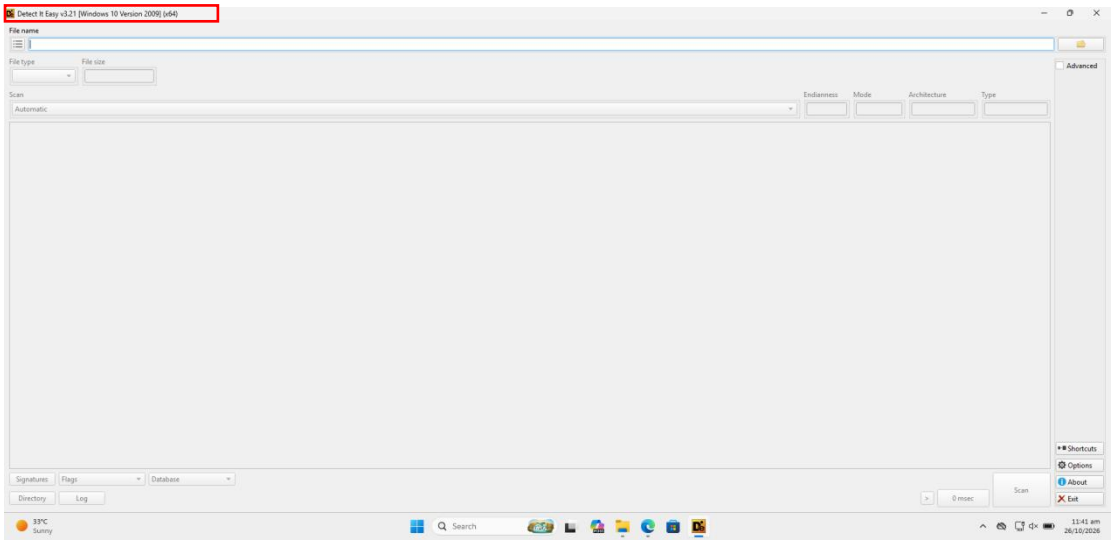


In the above figure the interface of **pestudio** is shown

2) To downlaod Detect it Easy tool visit the wesbite **softpedia**

Detect it easy: Uses to determine the file type, compiler, linker, architecture, entropy levels, Strings.

Link: https://www.softpedia.com/get/Programming/Packers-Crypters-Protectors/Detect-It-Easy.shtml



In the above figure the interface of **Detect it easy** tool is shown.

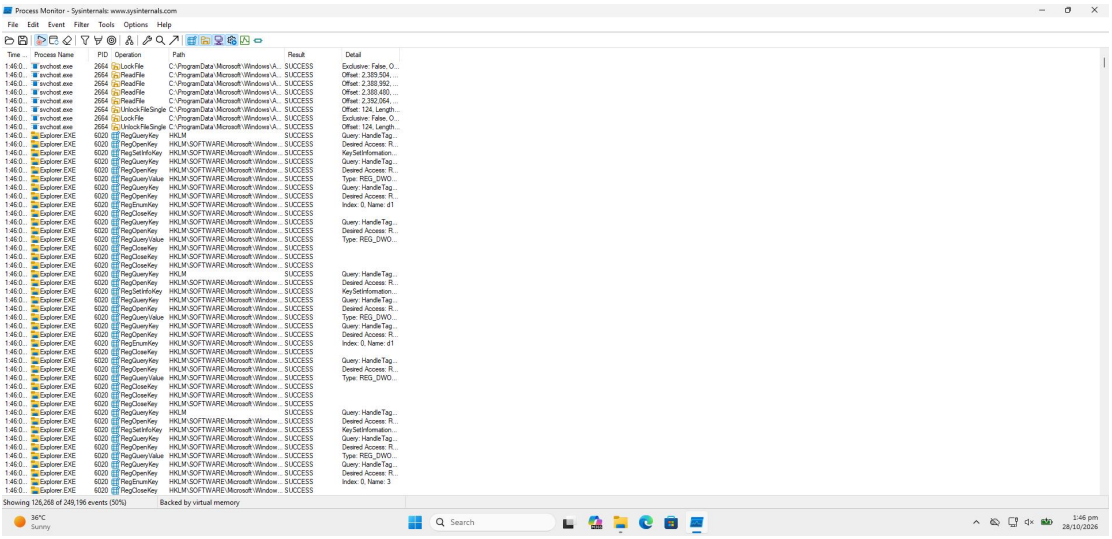
2) Dynamic Analysis Tools

The tools that has been used for **Dynamic analysis** here are Process Monitor (**procmon**),Wireshark, Regshot.

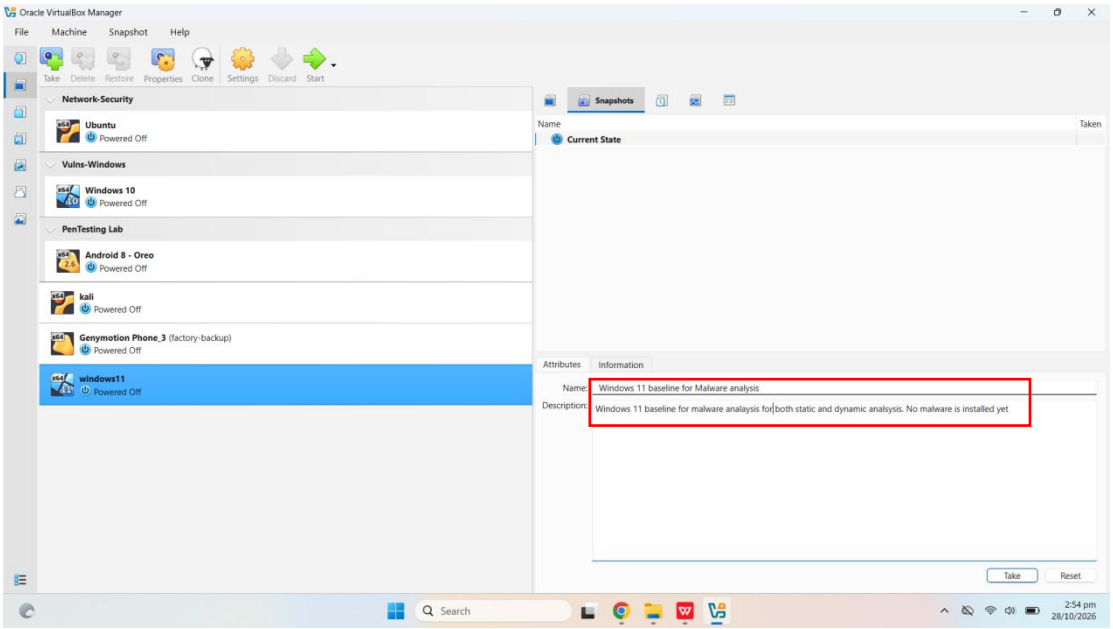
To download Process Monitor tool visit the website Microsoft sysinternals.

Link: https://learn.microsoft.com/en-us/sysinternals/downloads/procmon

Procmon: Used during dynamic analysis to monitor and record real-time Windows system activities, including file system changes, registry modifications, and process or thread creation performed by a running malware sample.

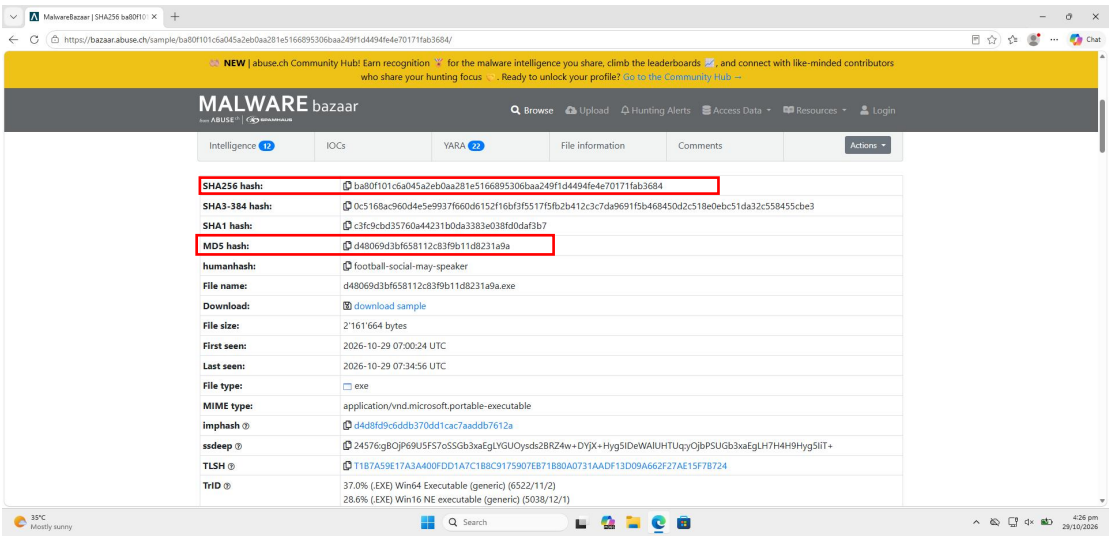


In the above figure the interface of **procmon** tool is shown.



In the above picture it is shown that a snapshot has taken before the Malware Analysis. The reason is to track all system modifications, registry changes, and artifacts generated during execution accurately. Furthermore, it allows the virtual machine to be reliably restored to its original clean state once the analysis concludes.

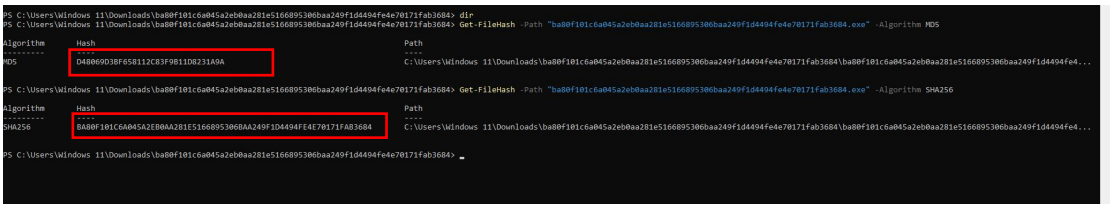
3) Sample Identification and File Hashes



In the above Picture it is shown that I have downlaoded Malware from MalwareBazar Platform. The file type of Malware is windows Portable executable file. The Hashes MD5 and SHA256 are mentioned below.

MD5 Hash: d4806d3bf658112c03f9b11d231a9a

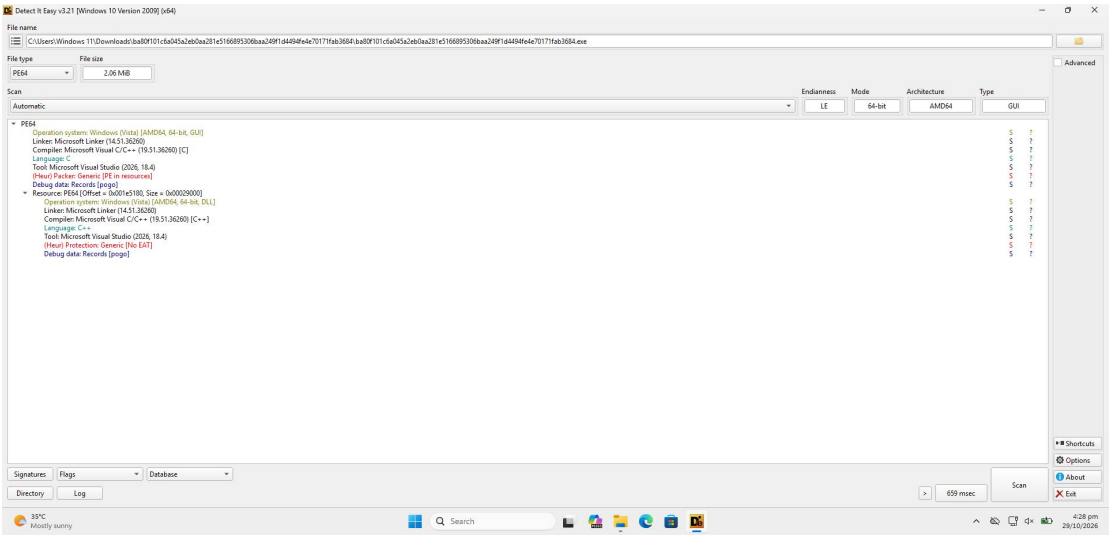
SHA256 Hash:
ba90f1f1c64045a2e0ae281e5166895306baa249f1d44941e4e70171fatc3684



In the above Picture it is demonstrated that execution of **PowerShell** commands to compute cryptographic hashes for the acquired malware sample. The Get-FileHash cmdlet was utilized to generate both **MD5** and **SHA256** checksums to verify file integrity and Both hashes get matched means file is orignal and authentic it has not been tampered by anyone.

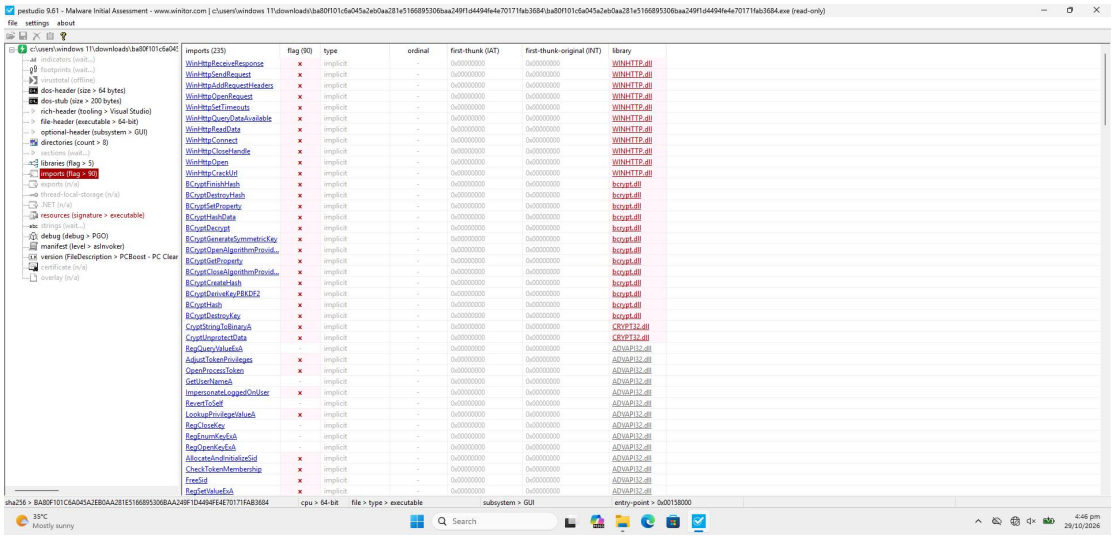
4) Static Analysis Findings

Static Analysis of the malware is the process of examining a suspicious file, script, or binary **without executing it**, allowing security analysts to safely study its code, structure, and metadata.

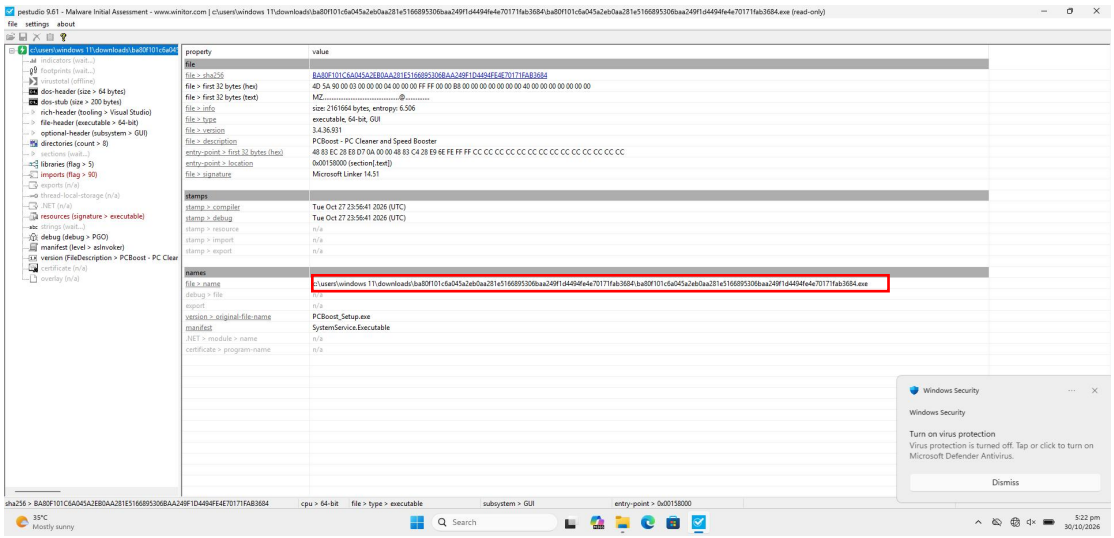


In the above Picture Detect it easy tool is used to identify the file Type and Metadata of file. The Scan reveals result in form of file type and metadata that has been breakdown below.

- **FileType**: PE64 (**Portable Executable 64-bit**)
- **Architecture**: AMD64, GUI, LE(Little-Endian)
- **Compiler**: Microsoft visual C/C++ compiled with Linker version 14.31.36300
- **Tool Used:** Microsoft Visual Studio (2022, 16.4)
- **Language:** C/C++
- **File Size:** ~2.06MB



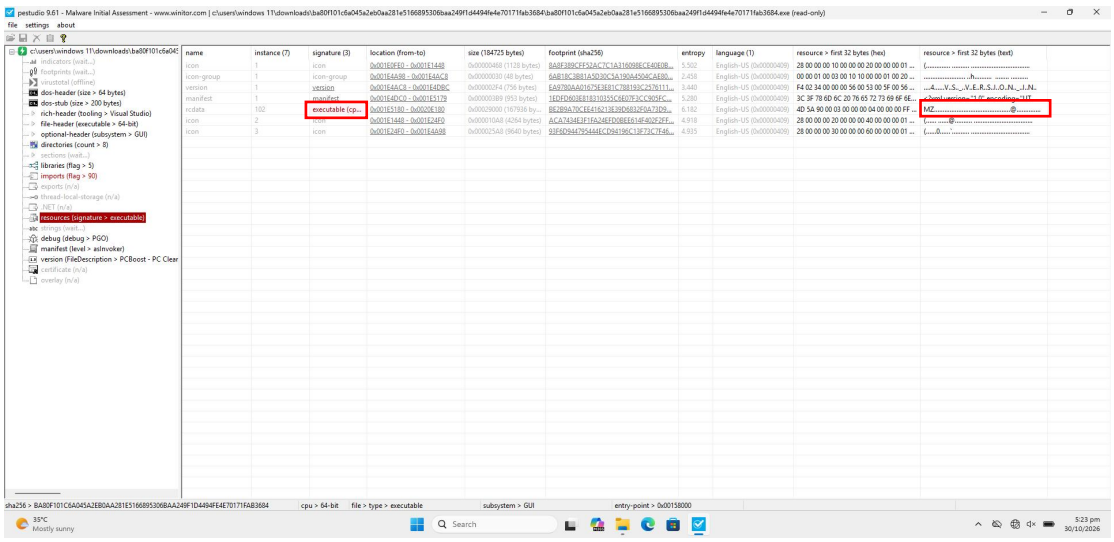
In the above figure, the PEstudio tool is used to analyze the binary's imported modules and functions, where loading a library merely grants access to external capabilities, but the actual malicious behavior stems from the specific API functions being called. Specifically, functions like **WinHttpSendRequest** and **WinHttpConnect** (WINHTTP.dll) enable the malware to establish outbound connections and exfiltrate data to a remote Command and Control (C2) server; **BCryptEncrypt** (bcrypt.dll) allows for data encryption to obfuscate stolen files or payload data; **CryptUnprotectData** (CRYPT32.dll) is heavily abused by credential stealers to decrypt saved passwords from Windows DPAPI; and **AdjustTokenPrivileges** (ADVAPI32.dll) is utilized for privilege escalation to bypass security controls. The combination of these specific API functions confirms that the binary is engineered for system reconnaissance, credential theft, and unauthorized network communication.



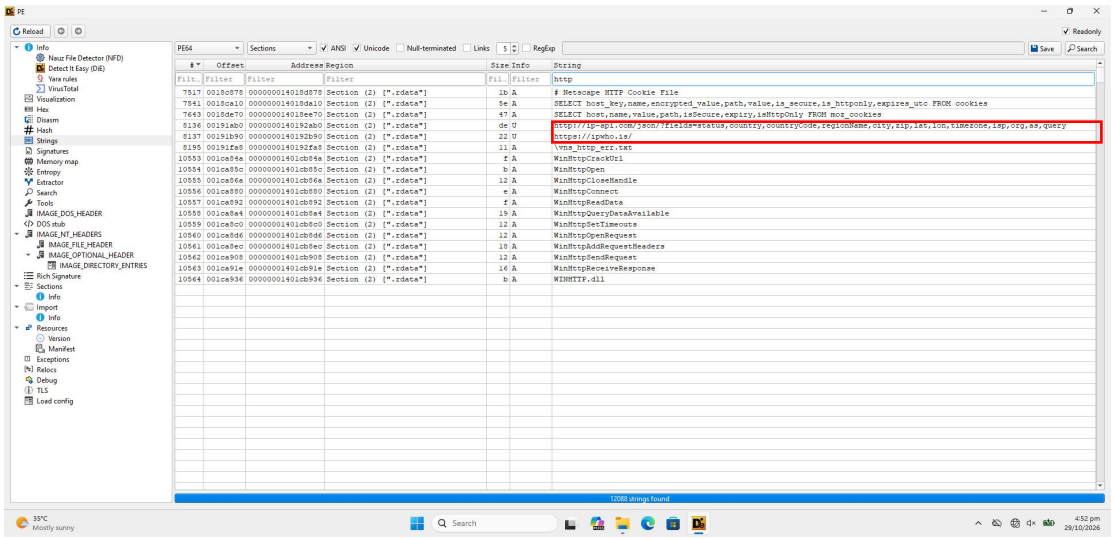
In the above Picture pestudio tool is used and we get the ule path where the malware is actually

C:\users\windows 11\downloads\ba80f1f1c64045a2e0ae281e5166895306baa249f1d44941e4e70171fab3684\ba80f1f1c64045a2e0ae281e5166895306baa249f1d44941e4e70171fab3684

.exe other than that critical masquerading techniques used by the author. Although the binary claims a file description of PCBoost - PC Cleaner and Speed Booster and an original file name of PCBoost_Setup.exe to appear legitimate to end-users, its internal manifest name (SystemService.Executable) and missing digital signatures confirm it is a suspicious binary disguised as system maintenance software.

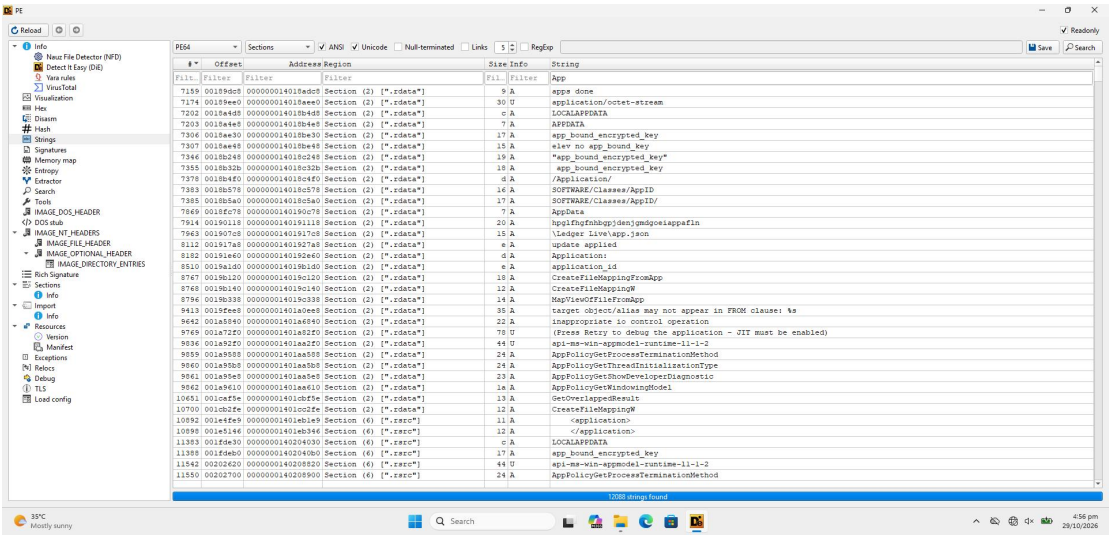


In the above figure, PEstudio is used to inspect the resources section of the binary, revealing standard embedded assets such as icons, version information , and the application manifest. However, a critical finding within the **rcdata** resources uncovers an embedded binary flagged with an executable signature that starts with the classic **MZ** header bytes **4D 5A**, confirming the presence of a nested executable payload hidden inside the resource section.



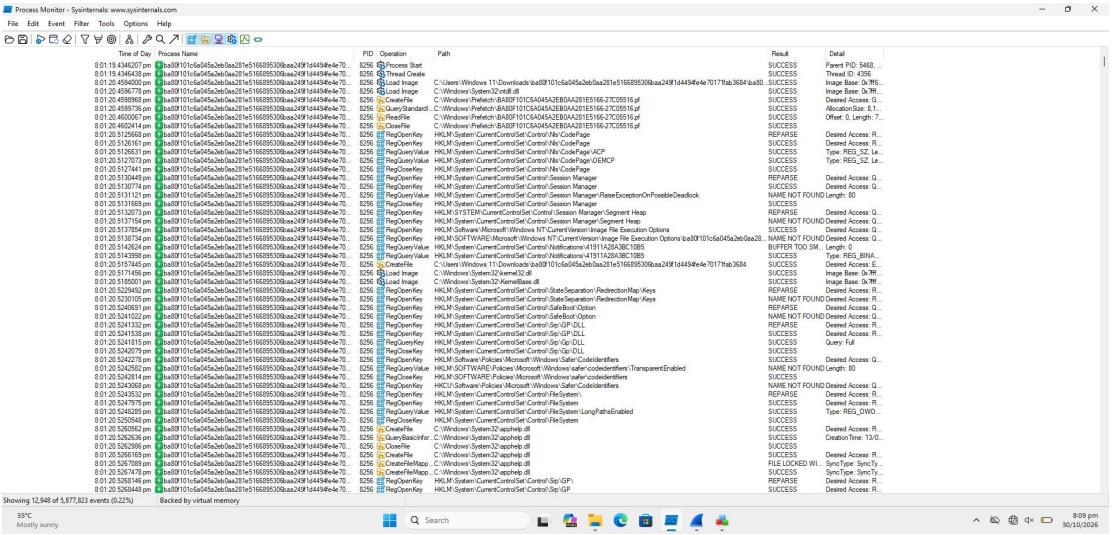
In the above picture **Detect it easy** tool **strings** functionality has utilized to identify any potentially suspsicious urls. The result give us 2 suspciious urls http://ip-api.com/json/?fields=status,country,countryCode,regionName,city,zip,lat,lon,timezone,isp,org,as,query and other one is https://ipwho.is/ are utilized by the malware to retrieve victim public IP addresses and geographic information. Further more in the above picture there is also an external SQL database query **SELECT host_key, name,**

encrypted_value, path, value, is_secure, is_httponly, expires_utc FROM cookies confirm that the sample actively extracts stored browser cookies and credentials.

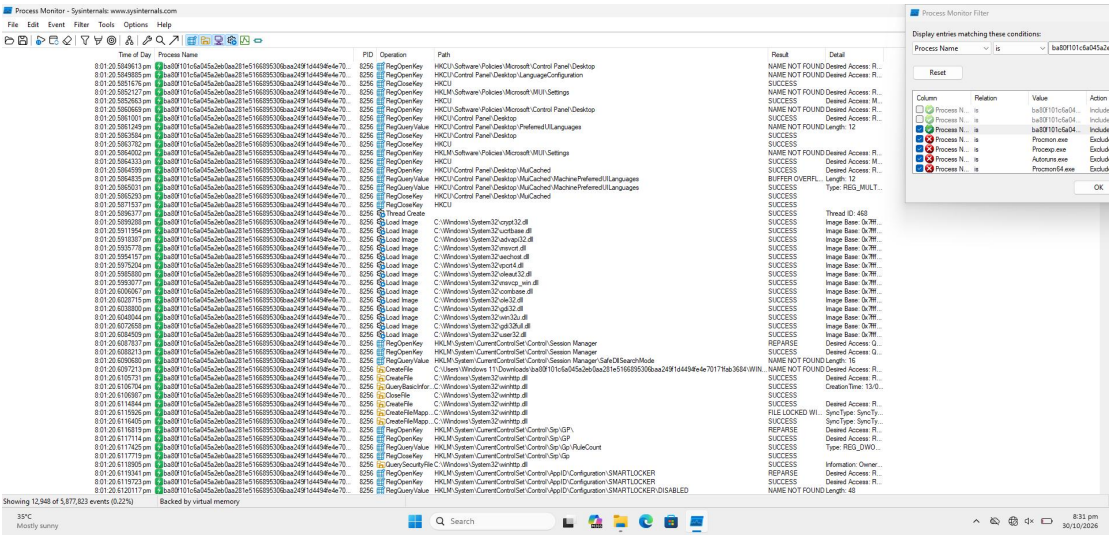


In the above picture **Detect it easy** tool **strings** functionality has utilized to examine printable strings. Strings revealing system environment references, targeted application paths, and runtime execution artifacts. Key findings include String references such as \Ledger Live\app.json, /Application/, SOFTWARE/Classes/AppID, and AppData indicate target locations for user data collection. Extracted strings like app_bound_encrypted_key and elev no app_bound_key point to internal routines designed to interact with application-bound encryption mechanisms

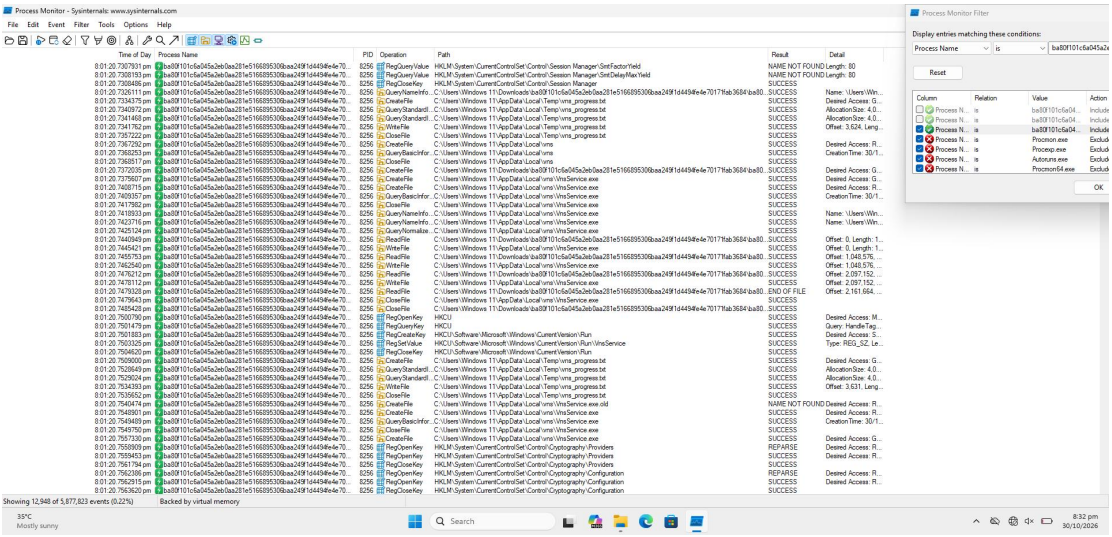
5) Dynamic Analysis Findings



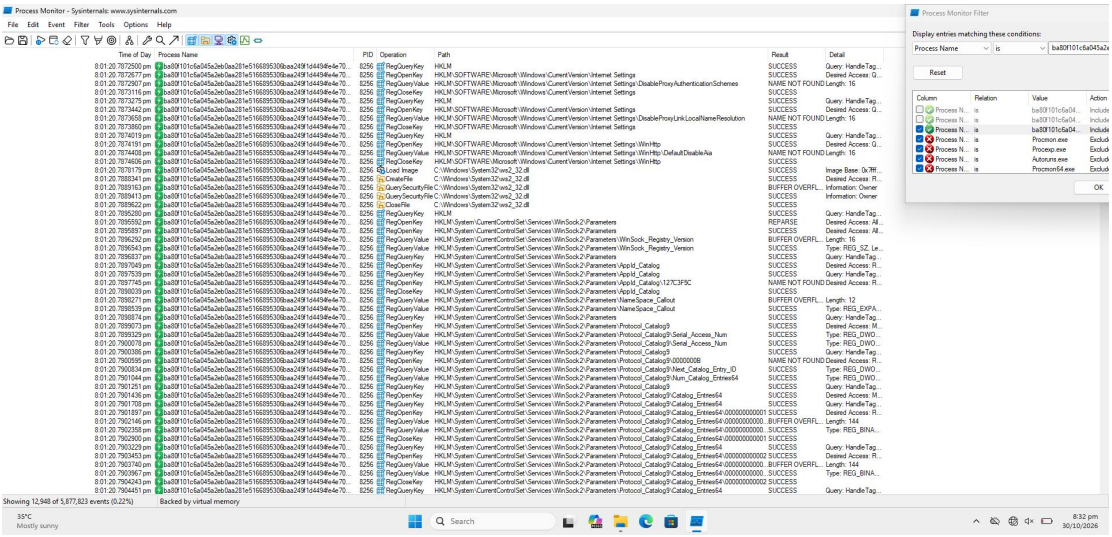
In the above figure it is shown that Procmon capture of process creation showing the Process Start and Thread Create events for the sample (PID 8256, Parent PID 5468), followed by loading of the executable image and ntdll.dll and creation of the Prefetch file, which confirms successful execution.



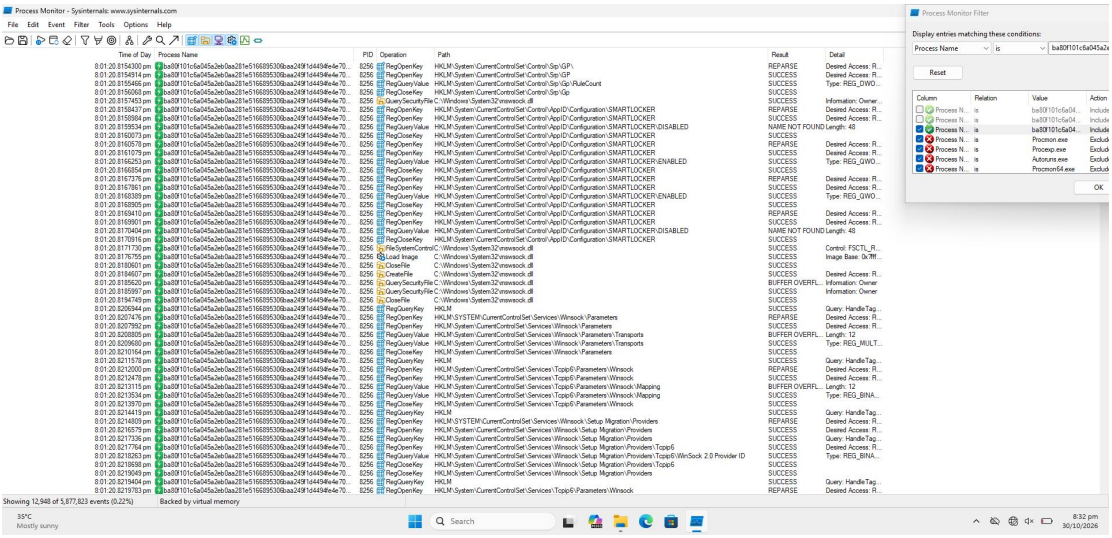
In the above figure it is shown that System language reconnaissance: the sample queries HKCU\Control Panel\Desktop (PreferredUILanguages and MuiCached\MachinePreferredUILanguages) to identify the victim's locale, then loads crypt32.dll, advapi32.dll and winhttp.dll, indicating cryptographic and HTTP communication capability



In the above figure it is shown that there is Dropper and persistence activit. the sample writes vns_progress.txt to the Temp folder, creates the AppData\Local\vns directory and copies itself to VnsService.exe using ReadFile/WriteFile operations, then creates the Run key value **HKCU\Software\Microsoft\Windows\CurrentVersion\Run\VnsService** (RegCreateKey, RegSetValue) so the copy executes at every user logon



In the above figure the Network preparation is shown, the sample queries Internet Settings and WinHttp proxy configuration, loads ws2_32.dll and enumerates the Winsock2 Protocol_Catalog9 and Catalog_Entries64 registry entries, initializing the Windows sockets stack ahead of outbound communication.



In the above figure there is Winsock transport initialization. mswsock.dll is loaded and the Winsock Transports and Tcpip6 Winsock Mapping registry values are queried, further confirming that the sample prepares TCP/IP networking, consistent with the C2 and data exfiltration capability identified during static analysis.

6) Indicators of Compromise

An **Indicator of Compromise (IOC)** is a piece of digital forensic evidence found on a computer, device, or network that serves as a highly reliable clue that a security breach or cyberattack has already occurred.

1) File based indicators

Indicator	Value	Why it is an IOC
SHA-256	ba80f101c6a045a2eb0aa281e5166895306baa249f1d4494fe4e70171fab3684	Unique fingerprint of the sample, used for detection and threat-intel lookups
Original file name	PCBoost_Setup.exe	Fake file name from the version resource, used to disguise the malware as a PC cleaner.
File Description	PCBoost PC Cleaner and Speed Booster	Masquerading as legitimate maintenance software
Internal manifest name	SystemService.Executable	Does not match the claimed product, which is a sign of a disguised binary
Digital Signature	Not present	Legitimate software of this kind is normally signed
Embedded Payload	MZ (4D 5A) executable in the rcdata resource	Nested executable hidden in the resources, which suggests dropper or loader behaviour

2) Host based Indicators

Indicator	Value	Why it is an IOC
Dropped Executable	C:\Users\Windows 11\AppData\Local\vns\VnsService.exe	Self-copy of the sample under a service-like name, created through ReadFile/WriteFile
Staging directory	C:\Users\Windows 11\AppData\Local\vns\	Directory created by the sample, which does not exist in a clean system.
Temp artifact	C:\Users\Windows 11\AppData\Local\Temp\vns_progress.txt	Progress/status file written during execution

3) Registry Indicators

Indicator	Value	Why it is an IOC
Run Key Value	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\VnsService	Makes the dropped copy start at every user logon.
Language queries	HKCU\Control Panel\Desktop\MuiCached\MachinePreferredUILanguages, PreferredUILanguages	Locale checks are commonly used for victim profiling

4) Network Indicators (Suspected from static analysis only)

Indicator	Value	Why it is an IOC
URL	https://ip-api.com/json/?fields=status,country,countryCode,regionName,city,zip,lat,lon,timezone,isp,org,as,query	Used to retrieve the victim's public IP address and geolocation, a common first step of infostealers before data exfiltration
URL	https://ipwho.is/	Alternative IP and geolocation lookup service, indicating the sample profiles the victim system.
Loaded modue	winhttp.dll	Provides HTTP/HTTPS communication capability, consistent with C2 communication and data upload.
Loaded module	ws2_32.dll, mswsock.dll	Windows sockets stack, loaded in preparation for outbound TCP/IP connections
Loaded module	crypt32.dll	Cryptographic and certificate functions, used for decrypting stored credentials and securing traffic.

5) Behavioural Indicators

Indicator	Value	Why it is an IOC
SQL Query	SELECT host_key, name, encrypted_value, path, value, is_secure, is_httponly, expires_utc FROM cookies	Reads the browser cookie database, which indicates theft of saved sessions and credentials.
Target Path	\Ledger Live\app.json	Targets the Ledger Live cryptocurrency wallet application data.
String	app_bound_encrypted_key, elev no app_bound_key	Related to Chrome app-bound encryption, suggesting a routine to bypass it and decrypt browser data,
API Import	CryptUnprotectData (CRYPT32.dll)	Decrypts DPAPI-protected data such as saved passwords.
API Import	BCryptEncrypt (bcrypt.dll)	Encrypts collected data before it is sent out.
API Import	WinHttpConnect, WinHttpSendRequest (WINHTTP.dll)	Establishes outbound connections and sends data to a remote server.
API Import	AdjustTokenPrivileges (ADVAPI32.dll)	Used to change process privileges, which can help bypass security controls.

7) Threat Assessment and Potential Impact

After Analyzing malware statically and dynamically This section assesses the risk the analysed sample (identified in this report as SalatStealer) poses to an organisation or an individual user. The assessment is based on behaviour **directly observed** during dynamic analysis in Procmon and on capabilities **inferred** from static analysis (imports, strings and embedded resources). Findings are labelled accordingly so that the reader can distinguish confirmed behaviour from probable behaviour.

7.1) Overall Risk Rating

Attribute	Assessment
Classification	Information stealer with dropper and persistence capability, delivered as a fake PC optimisation tool (PCBoost - PC Cleaner and Speed Booster).
Overall Severity	High
Likelihood of Compromise	High if executed by a user. The sample relies on social engineering (a fake utility) and requires no exploit.
Sophistication	Persistence uses a basic Run key and the disguise is a renamed utility, but the sample carries an embedded executable payload and decrypts browser data.

7.2) Assessed Capabilities and Supporting Evidence

The table below maps each assessed capability to the evidence collected in this investigation and to the MITRE ATT&CK framework to support triage and detection engineering.

Capability	Evidence	MITRE ATTACK	Confidence
Masquerading and user execution	Original file name PCBoost_Setup.exe, file description "PCBoost - PC Cleaner and Speed Booster", internal manifest name SystemService.Executable, no digital signature.	T1204.002 User Execution: Malicious File; T1036 Masquerading	High (static)
Dropper behaviou	MZ (4D 5A) executable embedded in the rcdata resource. Procmon shows the sample copying itself to AppData\Local\vns\VnsService.exe using ReadFile and WriteFile.	T1027 Obfuscated Files or Information; T1036 Masqueradin	High (observed)
Persistence	RegCreateKey and RegSetValue on HKCU\Software\Microsoft\Windows \CurrentVersion\Run\VnsService	T1547.001 Registry Run Keys / Startup Folder	High (observed)
Victim profiling	Queries of PreferredUILanguages and MuiCached\MachinePreferredUILan guages. Strings pointing to ip-api.com and ipwho.is for public IP and geolocation.	T1614.001 System Language Discovery; T1614 System Location Discovery	High (language, observed); Medium (geolocation static)
Credential and session theft	SQL string selecting host_key, name and encrypted_value FROM cookies. Imports of CryptUnprotectData and BCryptEncrypt. Strings app_bound_encrypted_key and elev	T1555.003 Credentials from Web Browsers; T1539 Steal Web Session Cookie	Medium (static)

	no app_bound_key.		
Privilege manipulation	Import of AdjustTokenPrivileges (ADVAPI32.dll).	T1134 Access Token Manipulation	Low (import only)
Command and control, exfiltration	WinHttpConnect and WinHttpSendRequest imports. winhttp.dll, ws2_32.dll and mswsock.dll loaded. Proxy and Winsock configuration queried in Procmon.	T1071.001 Web Protocols; T1041 Exfiltration Over C2 Channel	Medium (preparation observed, traffic not captured)

7.3) Potential Impact

Impact Area	Assessment
Confidentiality (High)	Theft of saved browser passwords, cookies and session tokens. Stolen session cookies can allow account takeover without the password and can bypass multi-factor authentication. Exposure of cryptocurrency wallet application data creates a risk of direct financial loss.
Integrity (Medium)	The sample writes a hidden copy of itself to a user profile folder and modifies the registry to survive reboots. The embedded payload also means additional components could be deployed after the initial infection.
Availability (Low)	No destructive or ransomware behaviour was observed. Direct availability impact is limited to the clean-up and rebuild of affected endpoints.
Individual	Loss of personal accounts, identity information and digital assets, followed by possible fraud or further phishing from the compromised accounts.

8) Detection and Mitigations Recommendation

8.1 Immediate Host-Based Containment (SOC Playbook)

1) **Process Termination:** Terminate any active process instance named VnsService.exe or running from %LocalAppData%\vns\ immediately.
2) **Persistence Removal:** Delete the malicious registry startup key using Command Prompt or PowerShell: REG DELETE "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "VnsService" /f
3) **Artifact Cleanup:** Remove dropped files and temporary staging directories from host endpoints:

%LocalAppData%\vns\VnsService.exe

%LocalAppData%\Temp\vns_progress.txt

8.2 Network-Level Controls & Egress Filtering

- **Domain & URL Blocking:** Block outbound communication at the perimeter firewall/proxy level for suspected geolocation and profiling services: ·

https://ip-api.com/json/?fields=status,country,countryCode,regionName,city,zip,lat,lon,timezone,isp,org,as,query

https://ipwho.is/

- **DNS Monitoring:** Monitor DNS query logs for unusual outbound connection requests originating from unverified binary executions.

8.3 Identity & Remediation Measures

- **Session Invalidation & Password Reset:** Revoke active web browser session tokens and force immediate password updates across accounts accessed on the impacted host.

- **Cryptocurrency Asset Recovery:** Transfer funds stored in software cryptocurrency wallets (e.g., Ledger Live) to non-compromised, fresh cold-storage addresses immediately.

9) Conclusion

The detailed static and dynamic analysis confirms that the analyzed executable is a malicious **SalatStealer** variant designed to compromise user credentials, browser session cookies, and financial assets. Through this hands-on investigation, I have learned and practically understood how real-world info-stealers operate and how to analyze them from a SOC analyst perspective:

- **Behavioral Tracking:** Dynamic execution monitoring via Process Monitor successfully validated key runtime behaviors, including dropping payload artifacts into %LocalAppData%\vns\, establishing persistent autorun capabilities via the registry Run key, and initializing Windows cryptographic and socket libraries.
- **Static and Dynamic Correlation:** I learned how to connect static indicators (such as imported APIs and strings) with actual runtime actions observed in the sandbox environment. Static inspection further confirmed routines targeting Windows DPAPI secrets (CryptUnprotectData), SQLite cookie databases, and targeted crypto-wallet data (such as Ledger Live).
- **Incident Response & Containment:** Practical analysis taught me how to identify Indicators of Compromise (IOCs) and formulate effective mitigation strategies. Applying the recommended host-based remediation steps, persistent registry key removal, and network egress rules will ensure complete containment and prevent further compromise across the enterprise infrastructure.